

AI integrated fuzzing

Geschreven door: Zacky Jarurattanon

Datum: 26-3-2026

Inhoudsopgave

Inhoudsopgave.....	2
Inleiding.....	3
De Aanleiding.....	3
Opbouw van het Rapport.....	3
Samenvatting van het Rapport.....	4
Methodologie.....	4
Fuzzing definitie.....	4
Methodologie: Implementatie en Experimentatie.....	4
1. Systeemconfiguratie en Omgeving.....	4
2. Corpus Generatie en Initiële Fuzzing.....	4
3. Monitoring en Visualisatie (Grafana).....	5
4. Probleemoplossing en Pivot (Appwrite).....	5
5. Data Pipeline Architectuur (ETL).....	5
Technische Competenties.....	6
Data Engineering.....	6
Data Science.....	7
Machine Learning.....	8
Modelkeuze: XGBoost (eXtreme Gradient Boosting).....	8
Mechanisme en Feature Selectie.....	8
Motivatie voor de keuze van XGBoost.....	8
Efficiëntie bij grote datasets.....	8
Ondersteuning voor sparse data.....	9
Invariantie voor feature-schaal.....	9
Target Selectie en Sampling.....	9
Professionele Ontwikkeling.....	10
Responsible AI.....	11
Randvoorwaarden en algemene regels.....	11
Model evaluatie.....	12
Toevoeging Model Evaluatie.....	12
Resultaten.....	13
Limitaties.....	14
Conclusie.....	15
Bronnen.....	15
Bijlagen.....	16

Inleiding

Fuzz-testing is een essentieel, maar complex domein binnen de softwarebeveiliging. Hoewel het concept op papier bedrieglijk eenvoudig lijkt — het systematisch invoeren van willekeurige of misvormde data om kwetsbaarheden te forceren — is de praktische uitvoering ervan een grote uitdaging. De enorme hoeveelheid data die hierbij wordt gegenereerd, vraagt om geavanceerde analysemethoden om patronen en zwakke plekken effectief te identificeren.

De Aanleiding

Dit stageproject is geïnitieerd en vormgegeven door het Data Science Centre of Excellence (DSCE). De focus ligt op het snijvlak van cybersecurity en kunstmatige intelligentie (AI). Waar traditionele fuzzing-methoden vaak handmatige analyse vereisen, onderzoekt dit project de mogelijkheden om dit proces te automatiseren en te verbeteren met behulp van machine learning.

De centrale hoofdvraag van dit onderzoek luidt dan ook:

"Tot hoeverre kan je ai gebruiken op een fuzzing output?"

Opbouw van het Rapport

In dit rapport wordt u meegenomen in het volledige proces: van de technische inrichting van de fuzzing-omgeving tot de uiteindelijke resultaten en de transformatie van de dataset.

Daarnaast vormt dit project de basis voor de verantwoording van mijn professionele ontwikkeling. De inhoud is gekoppeld aan drie specifieke Technische Competenties (TC). Om de leesbaarheid te vergroten en recht te doen aan de breedte van deze competenties, is de uitwerking hiervan onderverdeeld in drie subkolommen. Hierin wordt aangetoond hoe de verschillende aspecten van dit project — van Data Engineering tot Responsible AI — bijdragen aan het behalen van de leerdoelen binnen mijn opleiding.

Samenvatting van het Rapport

Methodologie

Fuzzing definitie

Voordat we beginnen, wat is fuzzing nou eigenlijk?

“Fuzz testing or fuzzing is an automated software testing method that injects invalid, malformed, or unexpected inputs into a system to reveal software defects and vulnerabilities. A fuzzing tool injects these inputs into the system and then monitors for exceptions such as crashes or information leakage. Put more simply, fuzzing introduces unexpected inputs into a system and watches to see if the system has any negative reactions to the inputs that indicate security, performance, or quality gaps or issues” (Black Duck, z.d.).

Methodologie: Implementatie en Experimentatie

In dit hoofdstuk wordt de technische opzet van het onderzoek beschreven. De methodologie volgt een iteratief proces van systeemconfiguratie, probleemoplossing en uiteindelijke data-generatie.

1. Systeemconfiguratie en Omgeving

Om de fuzzer **WuppieFuzz** operationeel te krijgen op de Virtual Machine (VM), is de omgeving als volgt ingericht:

- **Tooling:** Installatie van de programmeertaal **Rust** en de pakketbeheerder **Cargo**.
- **Software:** Het klonen van de WuppieFuzz core-repository en de bijbehorende dashboard-omgeving via GitHub.
- **Initialisatie:** Voor de eerste testfase is de **Petstore API** (v3) gebruikt als doelwittechnologie, geïmplementeerd via Docker ([swaggerapi/petstore3:unstable](#)). Er is gekozen voor de Petstore API vanwege de toegankelijke en heldere API-specificaties.

2. Corpus Generatie en Initiële Fuzzing

Een essentieel onderdeel van fuzzing is de 'corpus' (de verzameling input-data).

- Met behulp van het ingebouwde commando **WuppieFuzz output-corpus** is op basis van het **swagger.json** bestand een initiële set 'seeds' gegenereerd.
- De eerste succesvolle fuzzing-run werd uitgevoerd met een timeout van 120 seconden om de verbinding tussen de fuzzer en de target-API te valideren.

3. Monitoring en Visualisatie (Grafana)

Voor de analyse van de resultaten is een monitoring-stack opgezet met **Grafana**. Dit proces vereiste aanzienlijke troubleshooting:

- **Configuratie:** De standaard meegeleverde YAML-configuraties bleken niet compatibel met de VM-omgeving. Deze zijn handmatig aangepast in de Docker Compose-omgeving.
- **Data-integratie:** De dashboards zijn geïmporteerd via provisioning. Er is een handmatige SQLite-database ([report.db](#)) geïnitieerd binnen de VM, waarna de databron in Grafana is gekoppeld via het volledige bestandspad.

4. Probleemoplossing en Pivot (Appwrite)

Tijdens de vervolg-runs traden inconsistenties op, zoals stagnerende code-coverage (blijven hangen op 43/64 endpoints) en lege 'request bodies' in de database.

- **Interventies:** Er is getracht dit op te lossen door handmatige corpus-opschoning en het toevoegen van authenticatie-headers via een [headers.txt](#) bestand.
- **Pivot naar Appwrite:** Na herbeoordeling van de documentatie is besloten over te stappen op **Appwrite (versie 0.9.3)** als primair onderzoeksobject. Dit was vermeld in het tutorial van WuppieFuzz
- **Validatie:** Na overleg met de stagebegeleider en technische analyse van de rapportages is vastgesteld dat de "lege waarden" in de database geen fouten waren, maar het gevolg van inputs die door de fuzzer als niet-significant worden aangemerkt. Hierdoor is de gegenereerde dataset definitief als bruikbaar bestempeld voor het model.

5. Data Pipeline Architectuur (ETL)

De verwerking van de verzamelde data is ingericht volgens het **ETL-principe** (Extract, Transform, Load), uitgevoerd via drie gespecialiseerde Python-scripts samengevoegd in een bash script:

1. **Extract:** Ontsluiting van de ruwe data uit de SQLite-databasebestanden.
2. **Transform:** Opschoning, filtering en transformatie van de data naar een bruikbaar formaat voor machine learning.
3. **Load:** Het laden van de getransformeerde data in het uiteindelijke trainingsmodel.

Voor een gedetailleerde uitwerking van de technische implementatie van deze stappen wordt verwezen naar het hoofdstuk "Technische Competenties".

Technische Competenties

Data Engineering

Na een grondige Exploratory Data Analysis (EDA) is bepaald welke kenmerken relevant zijn voor de beantwoording van de onderzoeksvragen. Dit heeft geleid tot een gestructureerd proces van feature selectie en specifieke data-transformatiestrategieën.

1. Feature Selectie en Dimensionaliteit Reductie

De eerste stap bestond uit het identificeren van de relevante kolommen. Op basis van domeinkennis en de voorafgaande analyse bleek dat diverse kolommen redundant waren of geen direct meetbare signalen leverden voor het fuzzing-resultaat. De volgende kolommen zijn daarom verwijderd: 'id.1', 'url', 'testcase', 'data', 'error', 'timestamp.1', 'data.1', 'path' en 'reqid'. Deze uitsluiting is gebaseerd op de aanwezigheid van dubbele data, het ontbreken van unieke informatie of een te zwakke correlatie met de kernanalyse (zoals bij de curl-requests) en dat er 2 kolommen (data.1 en path) die zorgen voor dataleakage wat achteraf is gevonden tijdens het trainen van het model.

Daarnaast is een stappen filter toegepast om de datakwaliteit te verhogen. Om de volledigheid van de dataset te waarborgen, is besloten alleen volledige runs te behouden. Runs met minder dan tien tests zijn uitgesloten; dit garandeert dat de geanalyseerde campagnes het volledige beoogde bereik hebben doorlopen, wat een betrouwbaarder beeld geeft van het systeemgedrag.

2. Data-encoding en Categorical Variabelen

Voor verschillende categorische variabelen — specifiek 'body' en 'type' — was een transformatie naar numerieke waarden noodzakelijk. Omdat deze variabelen geen inherente ordinale volgorde kennen, is One-Hot Encoding (OHE) toegepast.

Een uitdaging hierbij was de hoge kardinaliteit (het grote aantal unieke waarden) van deze kolommen. Om te voorkomen dat de dimensie ruimte onbeheersbaar groot zou worden, is een limiet ingesteld: per categorische kolom zijn maximaal de 250 meest voorkomende unieke categorieën behouden, waarbij de overige waarden zijn gegroepeerd. Deze beperking was noodzakelijk om binnen de rekenkundige grenzen van het beschikbare virtuele machine (VM)-geheugen te blijven. De nieuwe dimensie is daardoor 598 numerieke features geworden.

3. Pipeline Implementatie en Efficiëntie

Vanwege het grote datavolume zou een directe verwerking van de gehele dataset leiden tot het overschrijden van het beschikbare VM-geheugen. Daarom is de pipeline ontworpen op basis van chunk-verwerking, waarbij de data wordt verwerkt in stabiele batches van 100.000 rijen per keer. Hoewel deze chunk-grootte variabel is, vormen 100.000 rijen momenteel de optimale balans tussen verwerkingssnelheid en systeemstabiliteit.

De pipeline is gerealiseerd middels een lichtgewicht Bash-script. De keuze voor Bash is ingegeven door het feit dat dit standaard aanwezig is op de VM, waardoor extra installaties overbodig zijn en de rekenkracht van de hardware nagenoeg volledig beschikbaar blijft voor de dataverwerking zelf.

Deze gestructureerde aanpak in feature selectie, filtering en batchverwerking heeft ervoor gezorgd dat de gehele dataset is gezuiverd en getransformeerd naar een bruikbaar formaat. Een direct resultaat van deze efficiënte transformatie is dat de omvang van de dataset is teruggebracht van circa 2 GB naar ongeveer 500 MB. Hiermee is de data niet alleen geoptimaliseerd voor opslag, maar ook volledig gereed voor verdere modellering en diepgaande analyse.

Data Science

Om een diepgaand inzicht te krijgen in de ruwe data en de geschiktheid voor modellering te beoordelen, is een uitgebreide Exploratory Data Analysis (EDA) uitgevoerd. Deze analyse vormt de basis voor de daaropvolgende data engineering-stappen.

1. Statistische Verkenning en Datastructuur De eerste fase van de EDA richtte zich op het vaststellen van de algemene structuur en kwaliteit van de dataset. Hierbij zijn de volgende statistieken en methodieken toegepast:

- **Missing Value Analysis:** Per kolom is de aanwezigheid van lege waarden (null values) gecontroleerd om de volledigheid van de dataset te bepalen.
- **Kardinaliteit Onderzoek:** Door het aantal unieke waarden per kolom te analyseren, is vastgesteld welke variabelen als categorisch of numeriek behandeld moeten worden en waar potentiële informatieoverlapping plaatsvindt.
- **Data-integriteit:** Tijdens de inspectie van de algemene dataset structuur zijn dubbele waarden (duplicates) geïdentificeerd. Deze dubbele vermeldingen zijn kritisch geëvalueerd om te bepalen of ze voortkomen uit systeem redundantie of meetfouten.

2. Opvallende Bevindingen en Outliers Een cruciale ontdekking tijdens de analyse betreft de stabiliteit van de verschillende test-runs. Door de doorloopsnelheid (throughput) te visualiseren, kwam naar voren dat **Run 13** een significante outlier is. Deze specifieke run vertoonde een afwijkend aantal requests per minuut in vergelijking met de overige runs. Deze afwijking is meegenomen in de verdere besluitvorming rondom data-opschoning om te voorkomen dat het model wordt beïnvloed door atypisch systeemgedrag.

3. Transparantie en Reproduceerbaarheid Voor een gedetailleerde weergave van de visualisaties, specifieke statistische verdelingen en de volledige code van deze analyse,

wordt verwezen naar het bestand [EDA.ipynb](#) in de GitHub-repository van dit project. Hierin zijn alle stappen van de dataverkenning gedocumenteerd en reproduceerbaar gemaakt.

Machine Learning

Modelkeuze: XGBoost (eXtreme Gradient Boosting)

Voor de analyse van de HTTP-requests is gekozen voor XGBoost. Dit is een krachtig ensemblemodel gebaseerd op gradient boosting met beslisbomen. Het model bouwt iteratief nieuwe bomen die specifiek gericht zijn op het corrigeren van fouten van voorgaande bomen.

Mechanisme en Feature Selectie

In tegenstelling tot sommige andere methoden wordt de variabelen belangrijkheid (*variable importance*) bij gradient boosting berekend door de verbetering in het splitsings criterium toe te kennen aan de variabele die gebruikt wordt bij iedere split in elke boom. Deze belangrijkheid wordt vervolgens geaggregeerd over alle bomen in het model (Hastie, Tibshirani, & Friedman, 2009, pp. 367–368). Hierdoor kunnen bepaalde variabelen sterker bijdragen aan het corrigeren van fouten, waardoor gradient boosting effectief een vorm van ingebouwde feature selectie toepast. Hoewel Random Forests eveneens gebruik maken van variabelen belangrijkheid op basis van splitsings criteria, merken Hastie et al. (2009, p. 593) op dat boosting-methoden sommige variabelen volledig kunnen negeren, terwijl random forests dit minder snel doen.

Motivatie voor de keuze van XGBoost

De keuze voor XGBoost boven alternatieve modellen, zoals een Support Vector Machine (SVM), is gebaseerd op verschillende technische en praktische overwegingen.

Efficiëntie bij grote datasets

De initiële dataset bevat bijna 880.000 rijen. Bij dergelijke volumes nemen de computationele kosten van SVM's sterk toe, mede door de kwadratische tijdscomplexiteit ($O(n^2)$) van veel kernel gebaseerde implementaties. XGBoost is daarentegen ontworpen voor efficiënte parallele verwerking en schaalbare training op grote datasets.

Ondersteuning voor sparse data

Zoals beschreven in het hoofdstuk *Data Engineering* bevat de dataset na transformatie 598 features. Door de toepassing van One-Hot Encoding bestaat een aanzienlijk deel van deze matrix uit nullen. XGBoost beschikt over ingebouwde ondersteuning voor sparse matrices, waardoor dergelijke data efficiënt verwerkt kan worden.

Invariantie voor feature-schaal

In tegenstelling tot afstand gebaseerde modellen zoals SVM vereist XGBoost doorgaans geen normalisatie of standaardisatie van features om goede prestaties te leveren. Dit vereenvoudigt de ontwikkelde data pijplijn (GeeksforGeeks, 2026a;2026b)

Target Selectie en Sampling

De doelvariabele van het model is de kolom status. Hiermee kan het model patronen herkennen in request-data die leiden tot een succesvolle afhandeling of een foutmelding. Vanwege een extreme klassen-onbalans (*class imbalance*) tussen de individuele statuscodes, is er op advies van collega's besloten om de verschillende HTTP-succescodes en foutcodes in de Machine Learning-code te combineren. Hierdoor is het probleem teruggebracht tot een binaire voorspelling: SUCCESS versus ERROR. Dit zorgt voor een stabielere trainingsproces en een betere focus op de hoofdtak: het vroegtijdig signaleren van mislukte verzoeken. En het zorgt ervoor dat de target kolom iets meer verdeeld is. Daarnaast om de imbalance te verminderen is er gekozen om class weights te gebruiken, wat het doet is class weights geeft de lagere klasse een hogere belang en bij hogere klasse een lagere belang, waardoor het model gedwongen wordt om meer aandacht te geven aan klassen die minder vaak voorkomt in het dataset (GeeksforGeeks, 2025).

Gezien de omvang van de gefilterde dataset (787.192 records) is een representatieve steekproef van 150.000 rijen genomen. Hierbij is *stratified sampling* toegepast op de kolom status, zodat de oorspronkelijke verhouding tussen succesvolle en mislukte verzoeken exact behouden blijft binnen de steekproef. Dit draagt bij aan de validiteit en betrouwbaarheid van het model. Waardoor de totale dimensie van de getransformeerde dataset 598 x 150.000 is. Om het model te trainen is er een train test split gebruikt in de verhouding 80/20, ofwel het model traint op 80% van het dataset en valideert dit op de overige 20%, daardoor wordt de dataset gesplit in 120.000 train rijen en 30.000 valideer rijen.

Daarnaast is gebruikgemaakt van *subsampling* om overfitting te verminderen. Hierbij is een waarde van 0,7 toegepast, wat betekent dat voor iedere boom willekeurig 70% van de trainingsdata wordt geselecteerd. Door niet steeds de volledige dataset te gebruiken, wordt de variatie tussen de bomen vergroot en neemt de kans op overfitting af.

Verder is gebruikgemaakt van de parameter `colsample_bytree`, eveneens ingesteld op 0,7. Deze parameter bepaalt welk deel van de beschikbare features willekeurig wordt geselecteerd voor het trainen van iedere boom. Wanneer de waarde kleiner is dan 1, wordt elke boom dus getraind op een willekeurige subset van de features. Dit kan de generalisatie

van het model verbeteren en vermindert de afhankelijkheid van specifieke features (Pyrowomat, 2018).

Als laatste zijn de parameters van het model aangepast zodat het werkt op systemen die een laag hardware specificatie hebben.

Professionele Ontwikkeling

Uitleggen hoe ik bijvoorbeeld de interviews heb gebruikt om kennis te vergaren over fuzzing. Uitleggen/link-leggen over de deskresearch die ik gebruikt heb. Feedback evaluaties en stagebegeleider. Hoe heb ik bijvoorbeeld omgegaan met tegenslagen.

Responsible AI

In dit onderzoek staan transparantie en verantwoorde dataverwerking centraal. Hieronder wordt de methodologie toegelicht, evenals de gehanteerde randvoorwaarden, evaluatie van het model en de juridische afbakening met betrekking tot de Europese AI-verordening.

Randvoorwaarden en algemene regels

1. Methodologie en Dataverzameling Voor het onderzoek is een fuzzing-campagne uitgevoerd met een totale looptijd van vier dagen. Gedurende de eerste drie dagen is er telkens twee uur gedraaid; op de vierde dag is de sessie uitgebreid naar drie uur om de diepgang van de resultaten te vergroten. Als doelwit (target software) is gekozen voor **Appwrite versie 0.9.3**. Dit is een verouderde versie met bekende kwetsbaarheden, wat het een geschikt object maakt voor het trainen van een model op basis van de getransformeerde resultaten uit de dataset.

2. Randvoorwaarden en Privacy (Data Governance) Om de integriteit en veiligheid van het onderzoek te waarborgen, zijn de volgende randvoorwaarden strikt gehanteerd:

- **Haalbaarheid en Scoping:** Om de tijdigheid van het project te garanderen binnen het gestelde tijdsbestek van vier maanden, is de scope van het onderzoek realistisch afgebakend. Er is bewust gekozen voor een haalbare complexiteit, zodat het project binnen de deadline succesvol afgerond kon worden.
- **Privacy-by-Design:** Er worden geen persoonlijk identificeerbare gegevens (PII) gedeeld of opgeslagen. In de configuratiebestanden (zoals `login.yaml`) zijn alle cookiegegevens en sessie-informatie structureel verwijderd om eventuele datalekken te voorkomen.
- **Openbare Opslag en Toegankelijkheid:** De resultaten en de broncode van het onderzoek worden ondergebracht in een openbare (publieke) GitHub-repository. Dit waarborgt de transparantie en reproduceerbaarheid van het onderzoek, terwijl het platform dient voor gestructureerd versiebeheer en een centrale opslag van de definitieve code.
- **Datagovernance en Verantwoordelijkheid:** De primaire verantwoordelijkheid voor de data binnen dit specifieke onderzoek ligt bij de hoofdonderzoeker. Bij een eventuele overdracht van het project wordt de verantwoordelijkheid voor de data-integriteit en het beheer officieel overgedragen aan de aangewezen opvolgers. Voor externe onderzoekers die via dit project hun eigen datasets genereren met *WuppieFuzz*, geldt dat zij zelf volledig verantwoordelijk zijn voor het beheer, de veiligheid en de integriteit van hun eigen data.

3. Juridische Afbakening: Europese AI-verordening Een essentieel onderdeel van de verantwoording is de toetsing aan de Europese AI-verordening (AI Act). Dit project valt buiten de reikwijdte van de AI-verordening, aangezien het onderzoek specifiek gericht is op **militaire toepassingen**. Volgens de richtlijnen van de verordening zijn systemen die uitsluitend voor militaire, defensie- of nationale veiligheidsdoeleinden zijn ontwikkeld of worden gebruikt, vrijgesteld van de hierin gestelde regels (Navigeren door de

AI-verordening, z.d.). Hiermee voldoet het onderzoek aan de geldende juridische kaders voor defensie-gerelateerd onderzoek.

Model evaluatie

Voor het evalueren van het model zijn de volgende metrics gekozen:

- F1-score uit het classificatierapport van Scikit-learn, specifiek de *macro average*
- Recall
- Confusion Matrix
- AUC ROC

De keuze voor deze metrics is gemaakt omdat zij samen een goed beeld geven van de prestaties van het model. Het classificatierapport van Scikit-learn toont verschillende evaluation metrics, waaronder precision, recall en F1-score per klasse. In dit onderzoek wordt voornamelijk gekeken naar de *macro average F1-score*. Deze metric berekent de gemiddelde F1-score over alle klassen, waarbij iedere klasse even zwaar meetelt. Hierdoor ontstaat een algemene indicatie van de prestaties van het model zonder dat grotere klassen een onevenredig grote invloed hebben op het resultaat.

Recall is gekozen omdat deze metric aangeeft hoeveel van de daadwerkelijk positieve gevallen correct door het model worden herkend. Binnen fuzzing is dit belangrijk, omdat gemiste crashes of foutieve classificaties kunnen leiden tot onontdekte kwetsbaarheden. Een hoge recall vermindert het aantal false negatives.

Daarnaast is de confusion matrix gebruikt omdat deze een gedetailleerd overzicht geeft van de voorspellingen van het model. De matrix toont het aantal true positives (TP), true negatives (TN), false positives (FP) en false negatives (FN). Op basis hiervan kunnen aanvullende analyses worden uitgevoerd, zoals het berekenen van accuracy, precision en recall. Hierdoor biedt de confusion matrix meer inzicht in de sterke en zwakke punten van het model.

Als laatste is de ROC-AUC-score (Area Under the Curve) gekozen als metric. Omdat het model een binaire classificatie uitvoert, biedt deze metric inzicht in hoe goed het model onderscheid kan maken tussen positieve en negatieve gevallen. Een hogere ROC-AUC-score duidt erop dat het model beter in staat is om beide klassen correct van elkaar te scheiden (GeeksforGeeks, 2026d).

Toevoeging Model Evaluatie

Voor dit model is bewust gekozen voor een verlaagde classificatiedrempel in plaats van de standaardwaarde. Deze keuze is ingegeven door de aanwezige class imbalance binnen de dataset, waarbij het aantal ERROR-cases aanzienlijk hoger ligt dan het aantal SUCCESS-cases. Bij gebruik van de standaarddrempel bestaat het risico dat het model minder gevoelig wordt voor het correct herkennen van de relevante minderheids- of kritieke klasse.

Door de drempelwaarde te verlagen van 0.50 naar 0.35 wordt het model gevoeliger voor het detecteren van foutieve gevallen. Dit leidt ertoe dat het model eerder als "ERROR" classificeert, waardoor de kans op het missen van daadwerkelijke fouten wordt verkleind. Binnen deze context is het voorkomen van gemiste fouten belangrijker dan het minimaliseren van foutieve detecties, waardoor recall als belangrijkste evaluatie-metric wordt beschouwd.

Deze keuze brengt echter een bewuste trade-off met zich mee. Een lagere drempel resulteert in een hoger aantal false positives, waardoor correcte SUCCESS-cases vaker onterecht als foutief worden aangemerkt.

Ondanks deze afweging blijft het model in staat om een duidelijk onderscheid te maken tussen de verschillende klassen. De algemene prestatie-indicatoren laten zien dat het model voldoende discriminatief vermogen bezit om binnen deze toepassing betrouwbaar ingezet te worden, waarbij de gekozen drempelwaarde aansluit bij de functionele doelstelling van het model.

Resultaten

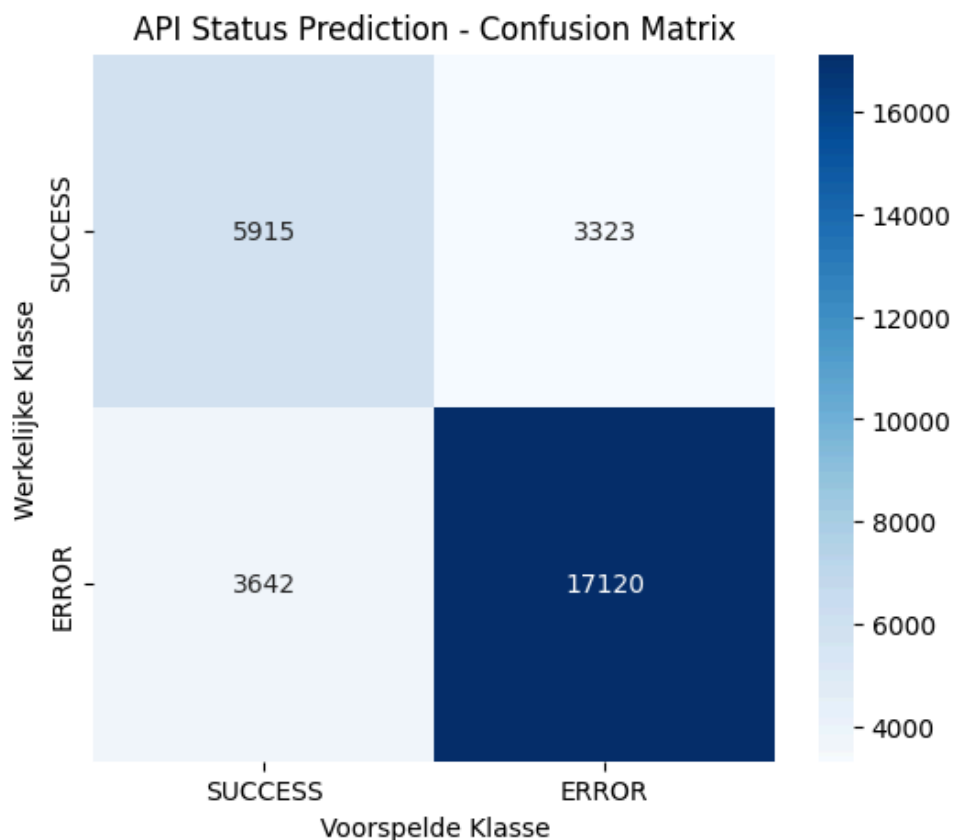
```
=== EVALUATION ===
Final AUC-ROC Score: 0.8529
-----

Classification Report:
              precision    recall  f1-score   support

    SUCCESS      0.62      0.64      0.63      9238
     ERROR      0.84      0.82      0.83     20762

 accuracy              0.77      30000
 macro avg              0.73      30000
 weighted avg           0.77      30000
```

Figuur 1: De evaluatie-metrics bevestigen dit beeld: met een **AUC-ROC van 0.85** laat het model een goede scheiding zien tussen beide klassen. De F1-scores (0.63 voor SUCCESS en 0.83 voor ERROR) geven aan dat het model duidelijk beter presteert op de meerderheidsklasse (*ERROR*), maar ook redelijke prestaties levert op de minderheidsklasse.



figuur 2: Het model presteert over het algemeen goed in het voorspellen van de API-status. De confusion matrix laat zien dat de meeste gevallen correct worden geclassificeerd, vooral de klasse *ERROR*. Wel zijn er nog fout-positieven en fout-negatieven, met name bij *SUCCESS*-gevallen die soms als *ERROR* worden voorspeld.

Limitaties

Tijdens het uitvoeren van dit onderzoek zijn er verschillende technische en infrastructurele beperkingen geobserveerd die invloed hebben gehad op de opzet en de omvang van de experimenten:

- **Hardwarematige restricties:** De beschikbare Virtual Machine (VM) beschikte over beperkte hardware-specificaties, te weten 8 GB RAM en geen grafische processor (GPU). Dit zorgde voor strikte computationele grenzen bij het verwerken van grote hoeveelheden data.
- **Aanpassing van de datasetgrootte:** Omdat de VM de initiële, omvangrijke dataset niet in zijn geheel kon verwerken zonder het geheugen te overschrijden, was het noodzakelijk om de dataset kunstmatig te verkleinen. Hierbij is gewerkt met een representatieve steekproef (sampled dataset) van 150.000 rijen. Daarnaast moest de dimensionaliteit worden ingeperkt door een limiet te stellen aan het aantal categorische kolommen (maximaal de 250 meest voorkomende categorieën per kolom).
- **Beperkt aantal test-runs:** Er zijn in totaal slechts 4 fuzzing-runs uitgevoerd. Deze beperking werd veroorzaakt door een initiële vertraging in de planning: het kostte één volledige stage maand om de fuzzer *WuppieFuzz* correct werkend te krijgen binnen de omgeving. Om de rest van het projectschema te bewaken, kon de definitieve data-generatie pas begin april starten nadat de problemen eind maart waren opgelost.
- **Suboptimale modelparameters:** Om te voorkomen dat de training van het XGBoost-model de VM-infrastructuur te zwaar zou belasten of zou laten vastlopen, zijn de model parameters specifiek aangepast aan systemen met lage hardwarespecificaties. Dit betekent dat het model mogelijk niet tot zijn maximale prestatiepotentieel is getuned.

Conclusie

Bronnen

Tno-S. (z.d.). GitHub - TNO-S3/WuppieFuzz-dashboard at c1feab829687e6d1e20d83d4d4824fa8d85b2ace. GitHub.
<https://github.com/TNO-S3/WuppieFuzz-dashboard/tree/c1feab829687e6d1e20d83d4d4824fa8d85b2ace>

Tno-S. (n.d.). GitHub - TNO-S3/WuppieFuzz: A coverage-guided REST API fuzzer developed on top of LibAFL. GitHub.

<https://github.com/TNO-S3/WuppieFuzz?tab=readme-ov-file>

<https://github.com/swagger-api/swagger-petstore?tab=readme-ov-file>

Navigeren door de AI-verordening. (z.d.). De Digitale Toekomst van Europa Vormgeven.

<https://digital-strategy.ec.europa.eu/nl/faqs/navigating-ai-act>

GeeksforGeeks. (2026a, 2 mei). Support Vector Machine (SVM) algorithm. GeeksforGeeks.

<https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/>.

Geraadpleegd op 12 mei 2026

GeeksforGeeks. (2026b, maart 19). XGBoost. GeeksforGeeks.

<https://www.geeksforgeeks.org/machine-learning/xgboost/>. Geraadpleegd op 12 mei 2026

GeeksforGeeks. (2026c, maart 30). Evaluation metrics in machine learning. GeeksforGeeks.

<https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/>.

Geraadpleegd op 12 mei 2026

GeeksforGeeks. (2026d, April 30). AUCROC curve in machine learning. GeeksforGeeks.

<https://www.geeksforgeeks.org/machine-learning/auc-roc-curve/>. Geraadpleegd op 4 juni

2026

GeeksforGeeks. (2025, 23 juli). How Does the class_weight Parameter in ScikitLearn Work?

GeeksforGeeks.<https://www.geeksforgeeks.org/machine-learning/how-does-the-classweight-parameter-in-scikit-learn-work/>.

Geraadpleegd op 26 mei 2026

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. Geraadpleegd op 18 mei 2026

Pyrowomat. (2018, June 26). subsample, colsample_bytree, colsample_bylevel in XGBClassifier() Python 3.x. Stack Overflow.

<https://stackoverflow.com/questions/51022822/subsample-colsample-bytree-colsample-bylevel-in-xgbclassifier-python-3-x>. Geraadpleegd op 19 mei 2026

What Is Fuzz Testing and How Does It Work? | Black Duck. (z.d.).

<https://www.blackduck.com/glossary/what-is-fuzz-testing.html>. Geraadpleegd op 29 mei

2026

Bijlagen

<https://github.com/WapixelXD/Stage-opdracht-2de-jaar/tree/main>